

FIAP X

Sistema de Processamento de Vídeos

Entregável — Pós-Graduação em Engenharia de Software · POSTECH 13SOAT

Data: 27/07/2026

1. Identificação dos participantes

Nome	Identificação (Discord)
Anderson Leonardo	Drayg
Oscar Souza	Khayzo

Tema: refatoração do projeto apresentado aos investidores — que processava vídeo dentro da requisição HTTP — para uma arquitetura distribuída, assíncrona e escalável, com autenticação, persistência, mensageria e observabilidade.

2. Links dos repositórios

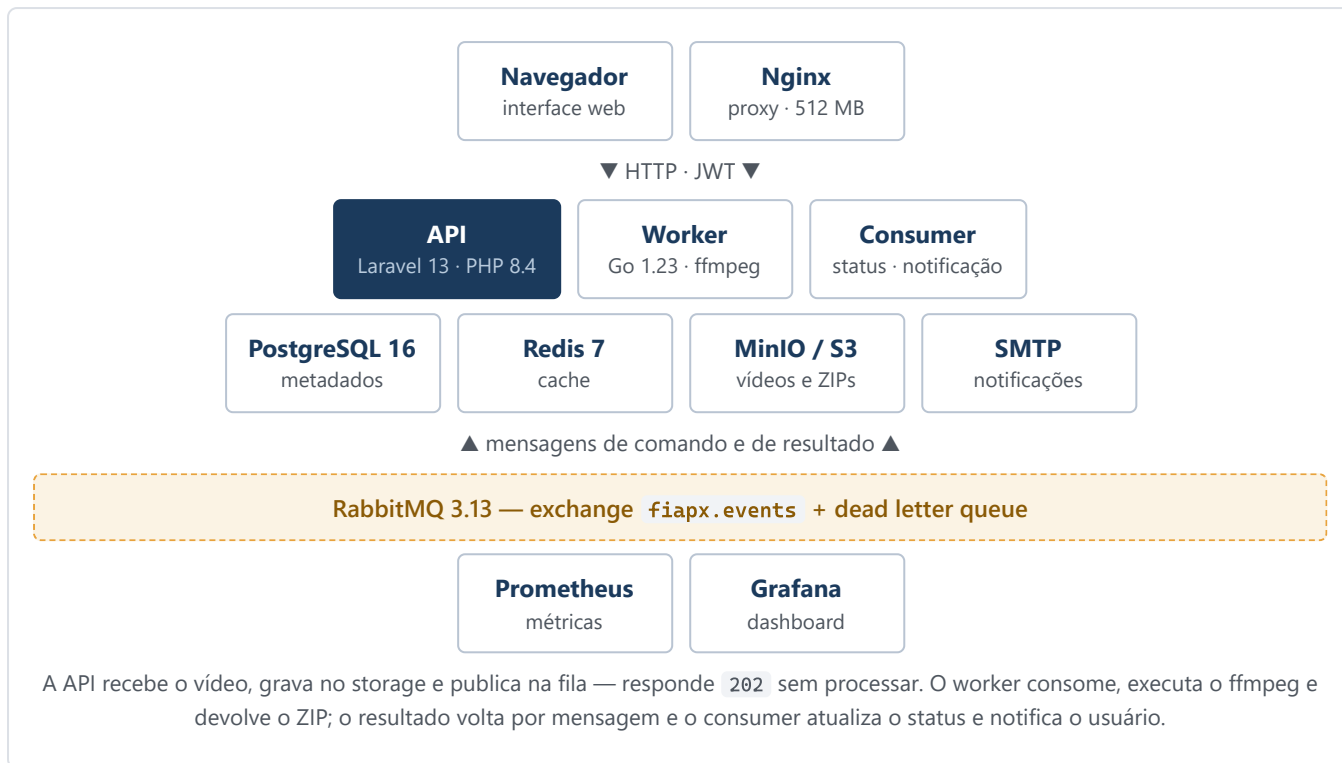
Componente	Repositório
Projeto completo API, worker, infraestrutura e documentação	github.com/Khayzoh/projeto_fiap_x

Repositório único contendo os dois serviços da aplicação, os manifestos de infraestrutura, a documentação arquitetural e o pipeline de CI/CD (`.github/workflows/`). O projeto base recebido está preservado em `legacy/` como referência do ponto de partida.

3. Vídeo de demonstração

Link (YouTube, até 10 minutos): <https://youtu.be/VH1eejSjL2c>

4. Diagrama geral da arquitetura



Comunicação: assíncrona via RabbitMQ entre API e worker (filas duráveis, mensagens persistentes, ack manual e DLQ) e síncrona via REST entre o navegador e a API. O download não passa pela aplicação: a API assina um link temporário e o cliente busca o arquivo direto no storage.

5. O desafio e a solução

O projeto recebido resolvia o caso de uso em um único arquivo, mas não sustentava operação real. Cada limitação foi endereçada por uma decisão arquitetural:

Limitação do projeto base	Solução adotada
Processamento síncrono dentro do request HTTP	Fila de mensagens: a API responde <code>202</code> e um worker separado processa
Arquivos em disco local	Storage de objetos (S3/MinIO) — serviços sem estado, escaláveis horizontalmente
Sem autenticação	Cadastro e login com bcrypt; JWT HS256 exigido em todas as rotas de vídeo
Sem banco de dados — estado inferido do disco	PostgreSQL com o ciclo de vida do vídeo e índices para a listagem por usuário
Sem fila — pico derruba requisição	Filas duráveis com mensagens persistentes; o excedente vira backlog, não erro
Sem testes	95 testes automatizados e gate de cobertura de 80% no CI

Tratamento de falhas

Os erros são classificados no domínio, e o consumidor age de acordo — o que evita tanto o laço infinito quanto a perda de trabalho:

Tipo	Exemplo	Comportamento
Transitório	storage indisponível, broker instável	Mensagem republicada com o contador incrementado; até 3 tentativas
Permanente	vídeo corrompido, job malformato	Vai para a <i>dead letter queue</i> , status <code>FAILED</code> e notificação ao usuário

Garantia contra perda em pico: exchange e filas duráveis, mensagens publicadas como persistentes, *publisher confirms* no worker, confirmação manual (`ack`) apenas após o processamento e limite de entrega por réplica (`prefetch`). Se o processo morrer no meio do ffmpeg, o job volta para a fila.

6. Justificativa da arquitetura e das tecnologias

A separação seguiu a **natureza da carga**, não o domínio: extrair frames é trabalho de CPU longo; receber upload e listar status é I/O curto. Separá-los permite escalar cada um pelo seu próprio gargalo.

Tecnologia	Onde	Justificativa
Go 1.23	Worker	Linguagem do projeto base — a extração foi aproveitada, não reescrita. Binário estático que sobe em segundos quando o autoescalonamento cria réplicas

PHP 8.4 + Laravel 13	API e consumidor	Reaproveita a base das fases anteriores: JWT, mensageria, logs, manifestos e pipeline
ffmpeg	Worker	Motor de extração herdado do projeto original, agora isolado atrás de uma interface
RabbitMQ 3.13	Entre os serviços	Desacopla receber de processar; filas duráveis e DLQ nativa sustentam o requisito de não perder requisição
PostgreSQL 16	API	Modelo relacional com integridade entre usuário e vídeo; gerenciado disponível em qualquer nuvem
Redis 7	API	Cache e sessão fora do processo. Não guarda status de vídeo — esse vem sempre do banco
MinIO / S3	Vídeos e ZIPs	Disco local impediria mais de uma instância; com storage de objetos os serviços ficam sem estado
Docker + Kubernetes	Todos	Mesma imagem em qualquer ambiente; HPA escala API (2→10) e worker (2→20)
Prometheus + Grafana	Observabilidade	Métricas de negócio, não só de infraestrutura; dashboard provisionado junto do projeto
GitHub Actions	Pipeline	Testes, cobertura, integração ponta a ponta e varredura de vulnerabilidades
JWT HS256	Autenticação	Validação local em cada serviço, sem round-trip; mantém a decisão da Fase 3

As decisões estão registradas em **5 ADRs** no repositório (docs/architecture/): processamento assíncrono por fila, worker em Go separado da API, storage de objetos no lugar do disco local, JWT com segredo compartilhado e a política de retentativas com DLQ.

7. Testes, qualidade e documentação de API

Módulo	Testes	Cobertura
API (Laravel)	64	83,5% de linhas
<code>worker/internal/domain</code>	31	90,9%
<code>worker/internal/usecase</code>		86,5%
<code>worker/internal/adapters/video</code>		84,8%
<code>worker/internal/config</code>		100%

- **Gate de 80%** verificado no CI — o pipeline falha abaixo disso.
- **Teste de integração ponta a ponta** (`scripts/smoke-test.sh`): sobe a stack completa e exercita limites de upload, cadastro, bloqueio sem token, três uploads simultâneos, download do ZIP e o caminho de erro até a DLQ e o e-mail.
- **Análise estática**: `go vet` e `gofmt` no worker, Laravel Pint na API.
- **Varredura de vulnerabilidades** com Trivy e validação dos manifestos Kubernetes com kubeconform.
- **OpenAPI** em `docs/api/openapi.yaml`, cobrindo autenticação, upload, listagem, detalhe, download e os endpoints de operação.

8. CI/CD e infraestrutura

Etapas	O que executa
Worker	<code>gofmt</code> · <code>go vet</code> · testes com <code>-race</code> · gate de cobertura · build do binário
API	Pint · PHPUnit com cobertura · gate de cobertura
Integração	sobe a stack com Docker Compose e roda o teste ponta a ponta
Qualidade	kubeconform nos manifestos · Trivy no código e nas imagens
Entrega	build das imagens, publicação no registro e deploy no Kubernetes com rollback automático em caso de falha

Manifestos Kubernetes (`k8s/`): namespace, ConfigMap e Secret, infraestrutura, API com sidecar de nginx e Job de migração, worker e consumidor, HPA, Ingress e PodDisruptionBudget. O `scaleDown` do worker é deliberadamente lento — derrubar uma réplica no meio do processamento custa reprocessar o vídeo inteiro.

9. Script de criação do banco de dados

O DDL completo está versionado em `database/schema.sql`, comentado e executável em um banco vazio:

- `users` — contas de acesso, com a senha em hash bcrypt.
- `videos` — ciclo de vida de cada vídeo, com UUID como chave (gerado pela API, viaja na mensagem e nomeia os objetos no storage), *constraint* de estado válido, chave estrangeira com `ON DELETE CASCADE` e

três índices que sustentam a listagem por usuário, o filtro por situação e a investigação por correlação.

- Tabelas de infraestrutura do framework (cache, sessões, filas e migrations).

Em execução, as tabelas são criadas pelas *migrations*, que são a fonte da verdade do schema. O arquivo `schema.sql` é o artefato documental equivalente, e foi validado aplicando-o em um banco descartável.

10. Atendimento aos requisitos

Requisito	Situação
Processar mais de um vídeo ao mesmo tempo	Atendido — N réplicas × prefetch
Não perder requisição em caso de pico	Atendido — filas duráveis + ack manual + DLQ
Sistema protegido por usuário e senha	Atendido — bcrypt + JWT
Listagem de status dos vídeos do usuário	Atendido — paginada, com filtro e isolamento por dono
Notificação ao usuário em caso de erro	Atendido — e-mail na falha e também na conclusão
Persistência dos dados	Atendido — PostgreSQL + storage de objetos
Arquitetura que permita escalar	Atendido — serviços sem estado + HPA
Projeto versionado no GitHub	Atendido — repositório único, com histórico completo
Testes que garantam a qualidade	Atendido — 95 testes + gate de 80%
CI/CD da aplicação	Atendido — GitHub Actions
Documentação da arquitetura	Atendido — diagramas, 5 ADRs e OpenAPI
Script de criação do banco	Atendido — <code>database/schema.sql</code>

Stack recomendada pelo enunciado: Containers (Docker + Kubernetes), Mensageria (RabbitMQ), Banco de dados (PostgreSQL + Redis), Monitoramento (Prometheus + Grafana) e CI/CD (GitHub Actions) — todos contemplados.